# CSE 816 — Software Production Engineering

## Mini-Project (10 Marks) — Problem Statement

*End-to-End CI/CD Pipeline for a Calculator REST API*

M.Tech (CSE) — Term I, 2026-27 | Stack: Java (Spring Boot + Maven)

**Faculty: Prof. B. Thangaraju**

| Item | Detail |
|---|---|
| **Project type** | Individual · same problem statement for all students |
| **Weightage** | 10 marks (10% of course) |
| **Assigned** | Class 16 — Mon, 28-Sep-2026 |
| **Work window** | Mid-term break (19-25 Oct) and surrounding weeks |
| **Submission & review** | Class 17 — Mon, 26-Oct-2026 |
| **Tools in scope** | Git/GitHub · Jenkins · Docker · Docker Compose (Modules 1-3) |

# 1. Objective

Build a small but production-style Calculator REST API in Java (Spring Boot), and wrap it in a complete, automated DevOps pipeline using only the tools covered before the mid-term: Git/GitHub for version control, Jenkins for continuous integration and delivery, Docker for containerization, and Docker Compose for deployment.

The application itself is intentionally simple — the marks are for the DevOps engineering around it, not for the complexity of the maths. By the end you should be able to push a code change to GitHub and watch it automatically build, test, package, containerize, and deploy with no manual steps.

> **Scope note:** This mini-project deliberately stops at Compose-based deployment. Ansible, Kubernetes, and Prometheus/Grafana are added later in the end-of-semester Capstone, which extends this same application into a full end-to-end framework.

## Learning outcomes

- Apply Git/GitHub branching and collaboration practices to a real codebase.
- Author a declarative Jenkins pipeline (Jenkinsfile) covering build, test, package, image build, and deploy.
- Write a clean, multi-stage Dockerfile following efficiency and security best practices.
- Orchestrate a multi-container deployment with Docker Compose.
- Connect the stages into a single automated, trigger-driven CI/CD workflow.

# 2. Application Specification

Implement a RESTful Calculator service using Spring Boot (Spring Web). The build must be Maven-based and produce an executable JAR.

## 2.1 Required REST endpoints

| Operation | Endpoint | Behaviour |
|---|---|---|
| Add | GET /api/v1/add?a=&b= | Returns a + b |
| Subtract | GET /api/v1/subtract?a=&b= | Returns a - b |
| Multiply | GET /api/v1/multiply?a=&b= | Returns a * b |
| Divide | GET /api/v1/divide?a=&b= | Returns a / b; b=0 → HTTP 400 with error JSON |
| Health | GET /actuator/health | Spring Actuator health probe (UP/DOWN) |

## 2.2 Functional requirements

- Accept numeric inputs and return a JSON response, e.g. `{ "operation": "add", "a": 2, "b": 3, "result": 5 }`.
- Validate inputs; return HTTP 400 with a structured error body for invalid/missing parameters and for division by zero.
- Expose Spring Boot Actuator so the container has a real health endpoint for Compose health checks.
- Include a minimal README documenting how to build, test, run, and call the API.

## 2.3 Testing requirements

- Unit tests with JUnit 5 for every operation, including the divide-by-zero case.
- At least one Spring MVC integration test (e.g. using MockMvc) hitting a real endpoint.
- Tests must run via `mvn test` and produce JUnit XML reports that Jenkins can publish.

# 3. DevOps Pipeline Requirements

## 3.1 Version control (Git / GitHub)

- Public or institute GitHub repository with a clear name and a complete README.
- Meaningful commit history (no single "final commit"); use at least one feature branch and a merge / pull request.
- A proper `.gitignore` for Java/Maven (exclude `target/`, IDE files, secrets).
- No credentials or secrets committed to the repository.

## 3.2 Continuous Integration & Delivery (Jenkins)

Create a declarative pipeline defined in a Jenkinsfile checked into the repository. The pipeline must be triggered automatically from GitHub (webhook preferred, Poll SCM acceptable) and include the following stages:

| Stage | What it must do |
|---|---|
| Checkout | Pull the latest code from GitHub on trigger |
| Build | Compile via `mvn clean package` (skipping tests here is fine) |
| Test | Run `mvn test`; publish JUnit results; fail the build on test failure |
| Docker Build | Build the Docker image, tagged with the build number |
| Push | Push the image to Docker Hub using Jenkins-managed credentials |
| Deploy | Deploy the stack with `docker compose up -d` and verify health |

- Configure email notification of build status (success/failure) to the student's email.
- Use the Jenkins Credentials store for the Docker Hub login — never hard-code it.
- The pipeline must be re-runnable and idempotent (a re-run must not break a running deployment).

## 3.3 Containerization (Docker)

- A **multi-stage Dockerfile**: a Maven/JDK stage to build the JAR, and a slim JRE runtime stage that copies only the JAR.
- Apply best practices: minimal base image, non-root user, explicit `EXPOSE`, layer caching for dependencies, and a `.dockerignore`.
- Image must run with a single `docker run` and respond on the mapped port.

## 3.4 Deployment (Docker Compose)

- A `docker-compose.yml` that brings up the calculator service.
- Define a network, a health check (using `/actuator/health`), restart policy, and environment variables.
- Add a second component to demonstrate Compose orchestration — for example an Nginx reverse proxy in front of the API, or a second replica behind the proxy.
- A single `docker compose up` must launch the full stack.

## 4. Suggested Repository Structure

```
calculator-devops/
  ├── src/main/java/...          # Spring Boot application
  ├── src/test/java/...          # JUnit tests
  ├── pom.xml                    # Maven build
  ├── Dockerfile                 # multi-stage build
  ├── .dockerignore
  ├── docker-compose.yml         # app + proxy/replica
  ├── Jenkinsfile                # declarative CI/CD pipeline
  ├── .gitignore
  └── README.md
```

## 5. Deliverables

1. GitHub repository URL with all source, Jenkinsfile, Dockerfile, and docker-compose.yml.
2. Docker Hub image link (public repository).
3. A short report (2-3 pages, PDF) covering: architecture diagram of the pipeline, description of each stage, and challenges faced.
4. Screenshots: a green Jenkins pipeline run, the email notification, `docker ps` showing running containers, and sample API responses.
5. A live demo in Class 17: push a change and show the pipeline auto-deploying it.

## 6. Evaluation Rubric (10 Marks)

| Criterion | Marks | Focus |
|---|---|---|
| Spring Boot API + JUnit tests (correctness, validation, coverage) | 2.0 | Application |
| Git/GitHub practices (branching, history, .gitignore, no secrets) | 1.0 | VCS |
| Jenkins CI/CD pipeline (all stages working, auto-trigger, notifications) | 3.0 | CI/CD |
| Dockerfile (multi-stage, best practices, runnable image) | 1.5 | Containers |
| Docker Compose deployment (multi-component, health checks) | 1.5 | Deployment |
| Report, screenshots & live demo | 1.0 | Docs/Demo |
| **Total** | **10.0** | |

## 7. Submission Guidelines & Policies

- Although the problem statement is common, every student must submit their own work; sharing of repositories or pipeline configurations will be treated as plagiarism.
- Commits must be made progressively — a repository created the night before submission with one commit will lose marks under Git practices.
- Use of AI assistants is allowed for learning, but you must understand and be able to explain every part of your pipeline during the demo.

- The deployment must actually run during the demo; screenshots alone are not sufficient for the deployment marks.

---

*This mini-project is the foundation for your end-of-semester Capstone, which will extend this same Calculator pipeline with Ansible, Kubernetes, and Prometheus + Grafana. Build it cleanly now and you will reuse most of it later.*